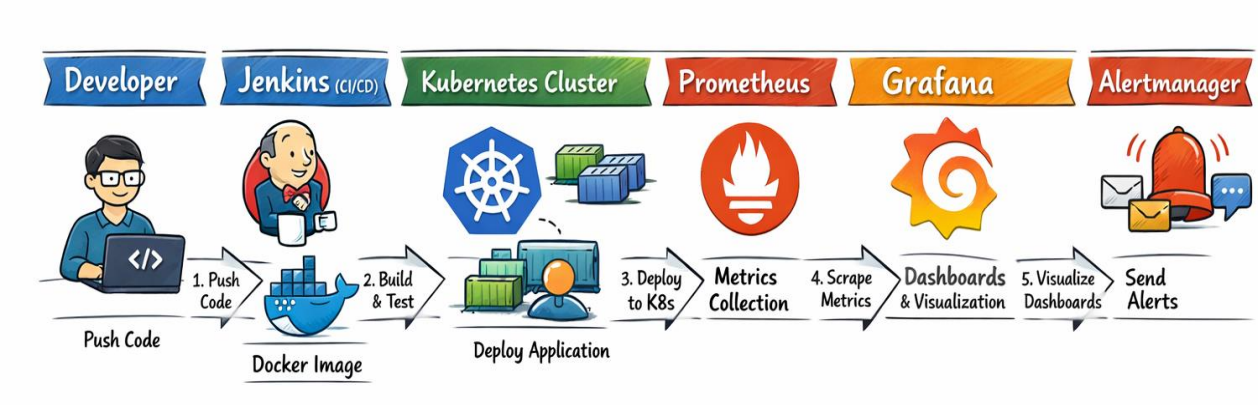
VLE-7: Full DevOps Monitoring System (AWS + K8s + CI/CD + Alerts)

🎯 Objective

To design and deploy an end-to-end DevOps monitoring system using:

- Amazon Web Services
- Kubernetes
- Jenkins
- Prometheus
- Grafana
- Prometheus Alertmanager

🏗️ System Architecture



📋 Prerequisites

- AWS account (Free tier or Educate)
- Ubuntu EC2 instance
- Basic knowledge of Docker & Kubernetes
- GitHub account

🔧 PHASE 1: Infrastructure Setup (AWS)

🔹 Step 1: Launch EC2 Instance

- Ubuntu 22.04
- t2.medium (recommended for K8s tools)
- Open ports:
  - 22 (SSH)
  - 3000 (Grafana)
  - 9090 (Prometheus)
  - 8080 (Jenkins)

🔹 Step 2: Install Required Tools

sudo apt update
sudo apt install docker.io git -y
sudo systemctl start docker
sudo usermod -aG docker ubuntu

🤖 PHASE 2: CI/CD using Jenkins

🔹 Step 3: Install Jenkins

sudo apt install openjdk-11-jdk -y
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ > /etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
Start:
sudo systemctl start jenkins

👉 Access:
http://<EC2-IP>:8080

🔹 Step 4: Configure Jenkins Pipeline

- Install plugins:
  - Docker Pipeline
  - Git

Sample Jenkinsfile:

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'docker build -t myapp .'
      }
    }
    stage('Deploy') {
      steps {
        sh 'kubectl apply -f deployment.yaml'
      }
    }
  }
}
```

☸️ PHASE 3: Kubernetes Setup

🔹 Step 5: Install Minikube (or use AWS EKS)

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube start

◈ Step 6: Deploy Sample Application

apiVersion: apps/v1
kind: Deployment
metadata:
name: myapp
spec:
replicas: 2
selector:
matchLabels:
app: myapp
template:
metadata:
labels:
app: myapp
spec:
containers:
- name: myapp
image: nginx
ports:
- containerPort: 80
Apply:
kubectl apply -f deployment.yaml

📊 PHASE 4: Monitoring using Prometheus & Grafana

◈ Step 7: Install Helm

sudo snap install helm --classic

◈ Step 8: Install Monitoring Stack

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack

---

◈ Step 9: Access Grafana

kubectl port-forward svc/monitoring-grafana 3000:80
Login:

- *admin / password (from secret)*

📊 Dashboards Include

- Node metrics
- Pod usage
- Cluster health

🔔 PHASE 5: Alerting using Alertmanager

🔹 Step 10: Configure Alerts

Example rule:

groups:

```
- name: k8s-alerts
  rules:
  - alert: PodCrashLoop
    expr: kube_pod_container_status_restarts_total > 3
    for: 2m
    labels:
      severity: critical
```

🔹 Step 11: Configure Alertmanager

- Email / Slack integration
- Alert routing

📩 Expected Alerts

- Pod failures
- High CPU usage
- Node down

🔄 PHASE 6: CI/CD + Monitoring Integration

🔹 Step 12: Full Flow

1. Developer pushes code → GitHub
2. Jenkins builds Docker image
3. Deploys to Kubernetes
4. Prometheus monitors pods
5. Grafana visualizes
6. Alert manager sends alerts

📊 Expected Output

- Live dashboards in Grafana
- Auto-deployment via Jenkins
- Alerts triggered on failures
- Multi-layer monitoring system

🎓 Sample Viva Questions

DevOps

1. What is CI/CD pipeline?
2. Difference between CI and CD?

Kubernetes

3. What is a pod?

4. What is deployment vs service?

Monitoring

5. What is Prometheus scraping?
6. Why Grafana is used?

Alerts

7. What is Alertmanager?
8. What are alert rules?

Result

Successfully designed and deployed a **Full DevOps Monitoring System** integrating:

- Cloud (AWS)
- CI/CD (Jenkins)
- Containerization (Docker)
- Orchestration (Kubernetes)
- Monitoring (Prometheus + Grafana)
- Alerting (Alertmanager)
